Methodology

Methodology I

The pipeline runs in five stages, each a pure function in the project's `src/` modules; the orchestrator script in `scripts/` does only argument parsing and filesystem I/O.

`infrastructure.prose.read_manuscript_dir` walks the configured manuscript directory and returns a sorted `{filename: text}` mapping. Scaffolding files (`preamble.md`, `config.yaml.example`, `AGENTS.md`, `README.md`, `SYNTAX.md`) are excluded by default — they hold infrastructure documentation rather than manuscript prose.

Analyse I

For each file, three analysers run in parallel:

Analyse I

- **Metrics** (`infrastructure.prose.analysis.metrics`) — word, sentence, paragraph, syllable counts; average words per sentence; average syllables per word; complex-word fraction; Flesch Reading Ease (Flesch 1948); Flesch-Kincaid Grade Level (Kincaid et al. 1975); Gunning Fog Index (Gunning 1952). The implementations are textbook formulae over a vowel-group syllable heuristic; they're good enough for a writer-facing signal, not for linguistic research.
- **Structure** (`infrastructure.prose.analysis.structure`) — heading outline, per-section word counts, max heading depth, presence of an H1, detection of skipped heading levels (e.g. an H1 followed directly by an H3).
- **Quality** (`infrastructure.prose.analysis.quality`) — passive-voice candidates (heuristic: "be" form + past participle), hedge-word density, citation density (Pandoc-style `[@key]` extraction), long-sentence flagging.

Analyse I

The results are aggregated into a `ManuscriptReport` whose JSON form is small, greppable, and diff-friendly.

Cross-check I

Every cited key is matched against the BibTeX file at `bibliography.references_path`. The check has four tunable behaviours via `config.yaml`:

Cross-check I

Setting	Effect
`fail_on_missing: true`	Any cited `[@key]` that does not exist in the bib fails the check.
`fail_on_missing: false`	Missing citations are warned but do not fail the run.
`fail_on_unused: true`	Bib entries that are never cited fail the check.
`fail_on_unused: false`	Unused entries are warned but do not fail.

Cross-check I

The check compares the cited keys from the `ManuscriptReport` against the BibTeX keys returned by `src.pipeline.prose_facade.parse_bib_keys` — a deliberately minimal regex that skips `@comment` blocks. (Forks that need dialect-complete BibTeX parsing can swap in `infrastructure.reference.citation.parse_bibfile` at the script layer; the exemplar's own gate does not require it.)

Evaluate I

The report runs through a set of pure check functions:

Evaluate I

- `_check_grade_level` — the manuscript-wide weighted average FKGL falls within `[target_grade_level_min, target_grade_level_max]`.
- `_check_citation_density` — citations per 1000 words $\geq$ `citation_density_min_per_1000`.
- `_check_no_skipped_levels` — no file uses a skipped heading level (when `prose.forbid_skipped_levels: true`).
- `_check_h1_per_file` — every file has an H1 (when `prose.require_h1_per_section: true`).
- `_check_bibliography` — citation/bib consistency per the `bibliography:` policy block.

Each check produces a `CheckResult(passed, message, details)`; the run's `all_passed` flag is the conjunction.

The thresholds each check applies come from `manuscript/config.yaml`. A `prose.preset` key (`lenient` for the bundled exemplar; `strict` in `config.yaml.example`) seeds the defaults for any knob the YAML does not set explicitly, so a fork can adopt a named editorial profile in one line and override individual knobs afterwards.

Render I

`src/manuscript/report.py::write_review_report` writes a markdown file with:

Render I

- Top-line counts (files, words, sentences, paragraphs, averages).
- A pass/fail table for every check.
- Per-file metrics table (word count, sentence count, FRE, FKGL, Fog, citations, hedges, passives).
- Heading outline per file.
- Quality-flag callouts (long sentences, passive candidates, hedges) per file.

Three orchestrators run in lexicographic order (so the figure and variable stages always see a fresh review):

Render I

- `scripts/run_prose_pipeline.py` writes `output/manuscript_report.json` (the raw `ManuscriptReport`), `output/checks.json` (the list of check results), `output/review_report.md` (the human-readable review report), `output/evidence_summary.json` (versioned diagnostic-only evidence), and `output/run_summary.json` (one-line metadata).
- `scripts/y_generate_prose_figures.py` reads `manuscript_report.json` and writes `output/figures/{section_word_counts,readability_metrics,citation_de`
- `scripts/z_generate_manuscript_variables.py` reads `manuscript_report.json`
  - `config.yaml` and writes `output/data/manuscript_variables.json` along with a resolved manuscript tree under `output/manuscript/`.
- `manuscript/references.bib` is **not** modified by this pipeline — the prose project does not generate citations, only validates them.